

Lesson 15: Saving Data Locally

Why save on the device

So far, your app forgets everything when it closes. The counter resets, the user's name is gone. Real apps remember: your theme choice, your login, your high score.

The simplest way to save small bits of data on the phone is `SharedPreferences`. It is a tiny key-value store: you save values with a name (the key), and read them back later -- even after the app restarts.

It is made for small things: settings, names, flags, counters. Not for big lists or files.

Saving a value

First add the package to `pubspec.yaml`:

```
dependencies:  
  shared_preferences: ^2.2.3
```

Then run `flutter pub get`.

Saving is three steps -- get the store, then save with a key:

```
final prefs = await SharedPreferences.getInstance();  
await prefs.setString('user_name', 'Aarav');  
await prefs.setInt('high_score', 150);  
await prefs.setBool('dark_mode', true);
```

There is a matching method for each type: `setString`, `setInt`, `setBool`, `setDouble`, and `setStringList`. See the `await`: saving takes a little time, so these methods are `async` (you met `await` in Lesson 5).

Reading it back

Reading uses the same keys. If nothing was saved yet, you get `null` -- so always have a fallback:

```
final prefs = await SharedPreferences.getInstance();  
final name = prefs.getString('user_name') ?? 'Guest';  
final score = prefs.getInt('high_score') ?? 0;
```

The `??` means "if the left side is `null`, use the right side". So a brand-new user sees "Guest" until they save a name.

A "remember my name" example

Here is the full pattern in a small widget:

```
class NameSaver extends StatefulWidget {
  const NameSaver({super.key});

  @override
  State<NameSaver> createState() => _NameSaverState();
}

class _NameSaverState extends State<NameSaver> {
  String name = 'Guest';
  final controller = TextEditingController();

  @override
  void initState() {
    super.initState();
    _loadName();
  }

  Future<void> _loadName() async {
    final prefs = await SharedPreferences.getInstance();
    setState(() {
      name = prefs.getString('user_name') ?? 'Guest';
    });
  }

  Future<void> _saveName() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('user_name', controller.text);
    setState(() {
      name = controller.text;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Hello, $name!'),
        TextField(controller: controller),
        ElevatedButton(
          onPressed: _saveName,
          child: const Text('Save'),
        ),
      ],
    );
  }
}
```

Two things to notice:

- initState() runs once when the widget is created. It is the right place to load saved data.
- After loading or saving, we call setState() so the screen shows the new value.

Close the app, open it again -- the name is still there.

Key terms

- SharedPreferences -- a small key-value store on the device for simple saved data.
- Key -- the name you save a value under, used to read it back later.
- initState() -- runs once when a stateful widget is created; a good place to load saved data.
- Fallback (??) -- a default value used when nothing was saved yet.